

Передовая инженерная школа
«Средства проектирования и производства электронной
компонентной базы»

Магистерская программа
«Вычислительные системы и ЭКБ»

Дисциплина «Микроконтроллеры и встраиваемые
системы»

Лабораторная работа 7



Аппаратные
таймеры

Симонов Сергей Борисович
доцент МПСУ МИЭТ, к.т.н.

Введение

Аппаратные таймеры – это периферийные устройства, которые позволяют генерировать временные отсчеты, измерять и формировать сигналы с высокой точностью. Поэтому таймеры имеют широкий спектр применения.

Цели работы

Познакомиться с функциональными возможностями аппаратных таймеров и научиться программировать таймеры в микроконтроллерах STM32 с помощью библиотеки HAL..

Подготовка и защита лабораторной работы

1. Для подготовки к выполнению и защите лабораторной работы необходимо:

- 1) Изучить настоящее пособие.
- 2) Подготовить черновые проекты решения практических заданий.
- 3) Ознакомиться с рекомендуемой литературой [1] и [2].
- 4) Проверить свои знания по перечню вопросов.

2. Защита лабораторной работы.

На защиту выносятся учебный проект в соответствии с практическим заданием. Самостоятельно разработанные программные решения должны компилироваться и запускаться без ошибок и предупреждений.

Во время защиты необходимо:

- продемонстрировать код и работу разработанных программ;
- ответить на теоретические вопросы и вопросы по представленному решению;
- выполнить дополнительное практическое задание по теме работы.

Литература и внешние ресурсы

- 1) UM2217 STM32 H7 LL Drivers.pdf, раздел 88 «HAL TIM Generic Driver»
- 2) RM0399 – stm32h745.pdf, раздел 41 « General-purpose timers (TIM2/TIM3/TIM4/TIM5)»
- 3) AN4013 – Introduction to timers for STM32 MCUs.
- 4) SR-04-Ultrasonic.pdf – HC-SR04 Ultrasonic Sensor Module.
- 5) RM0399 – stm32h745.pdf, раздел 16 « Direct memory access controller (DMA)»
- 6) DS - stm32h745zi.pdf, раздел 5 «Pin descriptions»

Теоретический материал

Часть 1. Аппаратные таймеры в STM32

1. Микроконтроллеры STM32 имеют несколько видов таймеров: базовые (Basic Timers), таймеры общего назначения (General Purpose) и продвинутые (Advanced Timers). Таймеры общего назначения включают в себя возможности базовых таймеров, но содержат расширенные функции. Аналогичным

образом продвинутые таймеры предлагают дополнительные функции к таймерам общего назначения. Все три вида таймеров поддерживают работу с контроллером DMA.



Также в семействах микроконтроллеров STM32 могут встречаться другие специальные таймеры, такие как таймеры пониженного энергопотребления LPTIM или таймер высокого разрешения (HRTIM).

2. Работа таймеров не зависит от работы процессора. Например, когда процессор приостанавливается во время отладки то таймер продолжает свою работу. Для конфигурации поведения таймеров в режиме отладки можно использовать макросы библиотеки HAL:

Макрос	Назначение
<code>__HAL_RCC_DBGMCU_CLK_ENABLE()</code>	Включить отладочный модуль
<code>__HAL_DBGMCU_FREEZE_TIMx()</code>	Задать для таймера <i>TIMx</i> остановку во время остановки процессора в отладке
<code>__HAL_DBGMCU_UNFREEZE_TIMx()</code>	Задать для таймера <i>TIMx</i> выполнение во время остановки процессора в отладке

3. Библиотека HAL предлагает программный интерфейс для работы с таймерами, который отражает аппаратную иерархию. При этом набор функций и структур зависит от типа таймера и режима работы (опрос, прерывание, DMA).

Кроме того, библиотека HAL предоставляет большое число полезных макросов для низкоуровневой работы с таймерами на уровне флагов регистров, например:

`__HAL_RCC_TIMx_CLK_ENABLE()` – включение тактирования;
`__HAL_TIM_ENABLE()` и `__HAL_TIM_DISABLE()` – для включения и отключения работы таймера;
`__HAL_TIM_GET_FLAG()` и `__HAL_TIM_CLEAR_FLAG()` – для проверки и сброса флагов прерываний;
`__HAL_TIM_CLEAR_IT()` – сброс бита ожидания обработки прерывания таймера (Pending bit).

Часть 2. Базовые таймеры

1. Базовые таймеры являются самым простым видом таймеров и предназначены для генерации временных отсчетов. В отличие от других таймеров они и не имеют выводов I/O. Структура базового таймера показана на рисунке 1.

2. События временных отсчетов могут обрабатываться в обработчике прерывания, быть источником запросов в контроллер DMA, триггером для цифро-аналогового преобразователя (ЦАП) или являться ведущим для других таймеров.

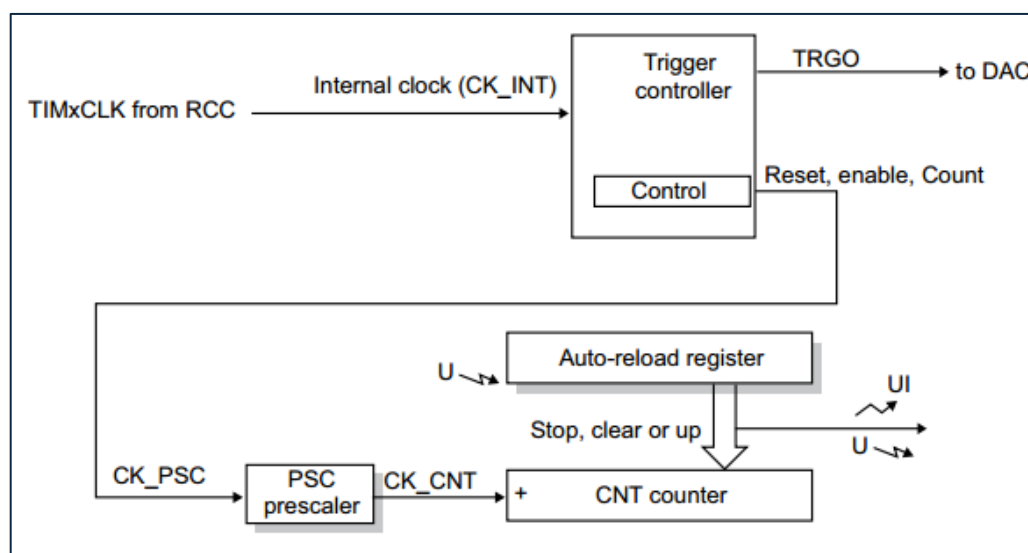


Рисунок 1 – Структура базового таймера STM32

3. Для работы таймера необходим тактовый сигнал CK_INT . Базовый таймер тактируется внутренним источником тактового сигнала, который поступает от блока RCC .

4. Каждый таймер имеет предделитель (*Prescaler*) – число, хранимое в регистре $TIMx_PSC$. После включения таймера путём установки флага CEN регистра $TIMx_CR1$ сигнал CK_INT (CK_PSC) поступает на вход предделителя и преобразуется в сигнал CK_CNT , который тактирует счетчик таймера.

5. Счетчик таймера хранится в 16-разрядном регистре $TIMx_CNT$.

6. Каждый таймер имеет значение перезагрузки, называемое также периодом таймера (*Period*), которое хранится в 16-разрядном регистре $TIMx_ARR$.

7. По фронту сигнала CK_CNT происходит увеличение счетчика таймера на единицу. Когда значение счетчика достигает значения регистра перезагрузки $TIMx_ARR$ то возникает событие переполнения и выполняются следующие действия:

- счетчик таймера $TIMx_CNT$ сбрасывается в ноль;
- генерируется событие обновления UE (*Update Event*);
- значение регистров $TIMx_ARR$, $TIMx_PSC$ обновляются по своим предзагруженным значениям;
- если разрешено перекрывание (флаг UIE в регистре $TIMx_DIER$), генерируется сигнал прерывания UIF ;
- если разрешен запрос DMA (флаг UDE в регистре $TIMx_DIER$), генерируется сигнал запроса к DMA .



Выше приведено общее поведение, которое используется в большинстве случаев применения базового таймера, но может быть изменено с помощью флагов регистров контроля таймера.

8. Частота возникновения события обновления UE базового таймера вычисляется по формуле:

$$f_{UE} = \frac{\text{Частота тактового сигнала таймера}}{(Prescaler + 1)(Period + 1)}$$

9. При работе с базовыми таймерами в библиотеке HAL используются следующие функции:

- $HAL_TIM_Base_Init()$ – для инициализации таймера;
- $HAL_TIM_Base_Start()$, $HAL_TIM_Base_Start_IT()$ и $HAL_TIM_Base_Start_DMA()$ – для запуска таймера в режима опроса, прирезываний и DMA .

Часть 3. Таймеры общего назначения

1. Функции таймеров общего назначения

1.1. Счетчик таймера общего назначения может быть как 16-разрядным ($TIM3$, $TIM4$), так и 32-разрядными ($TIM2$ и $TIM5$).

1.2. Таймеры общего назначения имеют различные режимы работы счетчика – *upcounting*, *downcounting* и *center-aligned* (см. рисунок 2).

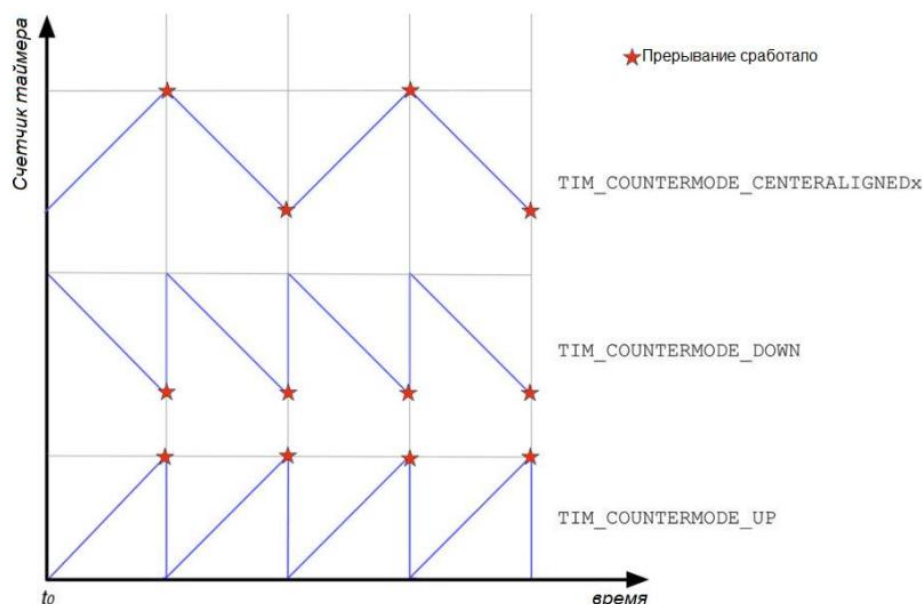


Рисунок 2 – Режимы работы счетчика таймеров общего назначения.

1.3. С таймером общего назначения могут быть связаны до 5 выводов I/O микроконтроллера: один пин ETR (External trigger input) может использоваться для внешнего тактирования таймера, а остальные пины каналов таймера (CH1, CH2, ...) – для внешнего запуска или тактирования таймера, а также для работы таймера в следующих режимах:

- режим сравнения входного сигнала (Input compare mode);
- режим сравнения выходного сигнала (Output compare mode);
- режим широко-импульсной модуляции (PWM mode);
- режим одиночного выходного импульса (One Pulse mode).

1.4. Таймер общего назначения может иметь до 4-х каналов. С каждым каналом связан регистр захвата/сравнения $TIMxCCRn$ такой же разрядностью, что и счетчик.



Здесь и далее по тексту в названиях регистров и флагов вместо n следует подставлять номер соответствующего канала таймера (от 1 до 4).

1.5. Также в таймеры общего назначения встроена аппаратная логика для обработки сигналов энкодера и датчиков Холла.

2. Синхронизация таймеров

2.1. Таймер общего назначения может работать как независимый, ведущий или ведомый таймер. Сигнал синхронизации, поступающий от ведущего таймера называют TRGO. Этот сигнал коммутируется по внутренним линиям МК на один из внутренних входов ITRx ведомого таймера. Для ведомого таймера этот сигнал называется сигналом триггерного входа TRGI.

2.2. Внутренние линии синхронизации таймеров приведены в таблицах «*TIMx Internal trigger connection*» (таблицы 353, 358, 361, 366 RM0399).

2.3. Таким образом программно активировав внутренние линии сигналов TRGO – TRGI между таймерами можно построить иерархические структуры таймеров (рисунок 3).

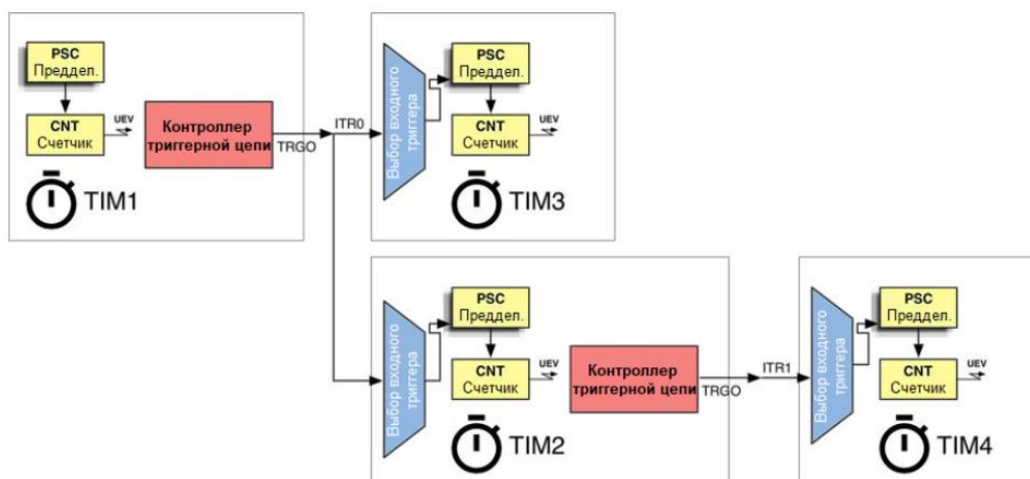


Рисунок 3 – Пример синхронизации таймеров

2.4. Возможные конфигурации синхронизации по выходу (TRGO) приведены в таблице:

Режим синхронизации по выходу	Условие генерации сигнала
По включению TIM_TRGO_ENABLE	Сигнал TRGO генерируется при включении ведущего таймера. Полезен для одновременного запуска нескольких таймеров или для управления временным окном, в котором ведомый таймер включается.
По обновлению счетчика TIM_TRGO_UPDATE	Событие обновления выбрано в качестве триггерного выхода (TRGO).
По событию захвата/сравнения TIM_TRGO_OC1	Триггерный выход посылает положительный импульс как только происходит событие захват или сравнения.
TIM_TRGO_OCnREF	Триггерный выход посылает положительный импульс как только на канале n происходит событие захват или сравнения.

2.5. Режимы синхронизации по входу приведены в следующей таблице:

Режим синхронизации по входу	Действие по сигналу
Режим сброса TIM_SLAVEMODE_RESET	Нарастающий фронт TRGI переинициализирует счетчик и генерирует обновление регистров.
Режим ворот TIM_SLAVEMODE_GATED	Тактирование счетчика активируются при высоком уровне и останавливается (без сброса счетчика) при низком уровне сигнала TRGI.
Режим триггера TIM_SLAVEMODE_TRIGGER	Счетчик запускается по нарастающему фронту сигнала триггерного входа. Управление происходит только запуском счетчика.
Режим внешнего тактирования TIM_SLAVEMODE_EXTERNAL1	Нарастающие фронты TRGI тактируют счетчик.

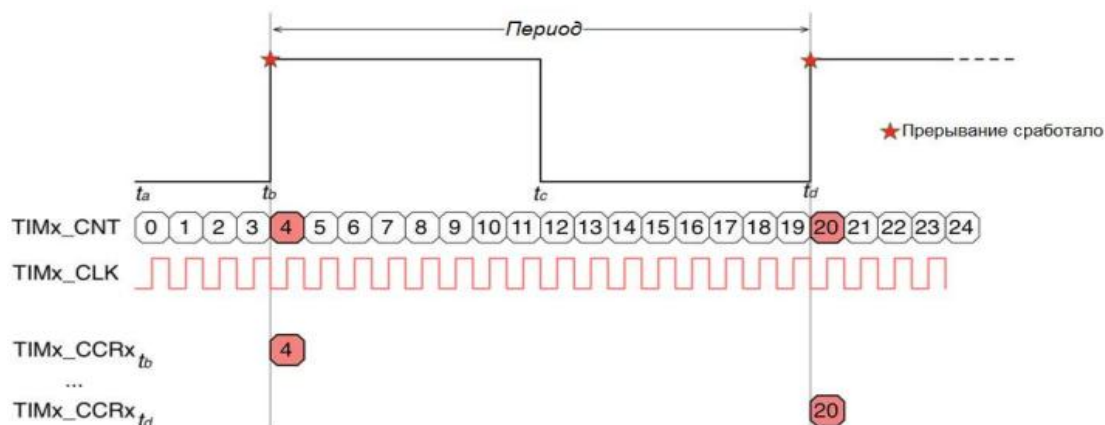
3. Режим захвата входного сигнала

3.1. Режим захвата входного сигнала устанавливается для каждого из 4-х каналов таймера отдельно (захват выполняется независимо для каждого канала).

3.2. Событие захвата поступает на входной каскад канала таймера, который содержит *детектор фронта* (edge detector) и *входной фильтр* (input filter), используемый для «борьбы со дребезгом» входного сигнала. Выход детектора фронта поступает в мультиплексор источников (IC1, IC2 и т. д.), который позволяет переопределить связь выводом МК и каналов. Также имеется *предделитель* канала, который позволяет снизить частоту входного сигнала.

3.3. В момент захвата в регистр захвата/сравнения $TIMx_CCRn$ копируется счетчик таймера, выставляется флаг $CCnIF$ и, опционально, генерируется прерывание и запрос DMA. Генерация запроса и прерывания управляется флагами $CCnIE$ и $CCnDE$ соответственно.

3.4. Режим захвата предназначен для измерения периода сигнала, например, как показано на рисунке 4.



$$\text{Период} = \text{Захват} \cdot \left(\frac{TIMx_CLK}{(\text{Prescaler} + 1)(\text{Предделитель канала})(\text{Индекс полярности})} \right)^{-1}$$

$$\text{Захват} = CNT_1 - CNT_0, \text{ если } CNT_0 < CNT_1$$

$$\text{Захват} = (TIMx_Period - CNT_0) + CNT_1, \text{ если } CNT_0 > CNT_1$$

Индекс полярности = 1 (захват по возрастающему фронту) или 2 (захват по обоим фронтам).

Рисунок 4 – Измерения периода тактового сигнала в режиме захвата

4. Режим сравнения-вывода (Output Compare Mode)

4.1. Режим *Output Compare* позволяет управлять сигналами на выходных каналах.

4.2. Когда значение регистра захвата/сравнения канала ($TIMx_CCRn$) совпадает со счетчиком таймера ($TIMx_CNT$) выполняются следующие действия:

- устанавливается флаг $CCnIF$
- генерируется прерывание и запрос DMA, если установлены соответствующие биты разрешения в регистре $TIMx_DIER$.
- устанавливается сигнал на выводе выходного канала в соответствии с выбранным режимом и полярностью.

4.3. Режимы управления выходным каналом приведены в таблице:

Режим сравнения и выхода	Действия при совпадении счётчика с регистром сравнения
Временной отсчет <i>Output compare timing</i>	Вывод канала не изменяется.
Активация <i>Output compare active</i>	Вывод канал устанавливается высокий уровень.
Деактивация <i>Output compare inactive</i>	Вывод канал устанавливается низкий уровень.
Переключение <i>Output compare toggle</i>	Вывод канала инвертируется.
Принудительное значение <i>Output compare forced active/inactive</i>	Вывод канала принудительно устанавливается в высокий или низкий уровень независимо от значений счетчика.

5. Режим широко-импульсной модуляции (PWM Mode)

5.1. Пример сигнала широтно-импульсной модуляции (ШИМ, англ. *Pulse-width modulation*, PWM) приведен на рисунке 5.

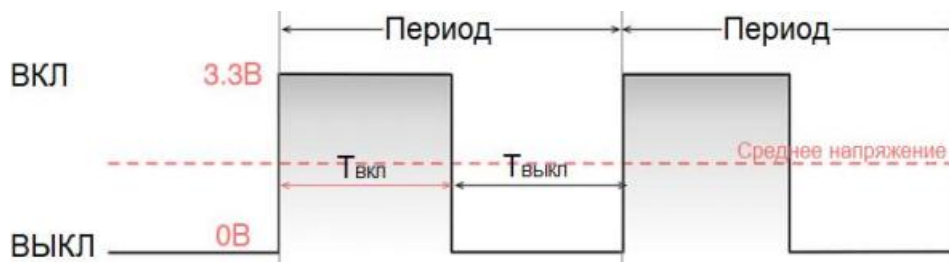


Рисунок 5 – Сигнал ШИМ с коэффициентом заполнения 50%.

5.2. К основным параметра сигнала ШИМ относятся период (частота) и длительность импульса ($T_{вкл}$), а также коэффициент заполнения, который определяется по формуле:

$$D = \frac{T_{вкл}}{Период} \times 100\%$$

5.1. Сигнал ШИМ имеет множество применений в цифровой электронике. Например, при фильтрации ШИМ-сигнала можно построить цифро-аналоговый преобразователь (изменяя коэффициент заполнения сигнала, можно регулировать выходное напряжение); с помощью силовых транзисторов можно управлять большими токами и нагрузкой.

5.2. Таймер в режиме ШИМ позволяет формировать на выходе канала ШИМ-сигнал и динамически изменять его параметры.

5.3. Принцип работы таймера в режим ШИМ аналогичен режиму сравнения-вывода. Период сигнала ШИМ записывается в регистр перезагрузки таймера $TIMx_ARR$ (параметр *Period*). Длительность импульса записывается в регистр сравнения $TIMx_CCRn$ (параметр *Pulse*). Форма выходного сигнала меняется в соответствии выбранном режимом ШИМ.

Режим ШИМ	Действие при совпадении счётчика со значением регистра сравнения $TIMx_CCRn$
PWM mode 1	При прямом отсчете вверх канал активен, пока $Period < Pulse$, иначе неактивен. При обратном отсчете канал неактивен, пока $Period > Pulse$, иначе активен.
PWM mode 2	При прямом отсчете вверх канал не активен, пока $Period < Pulse$, иначе активен. При обратном отсчете канал активен, пока $Period > Pulse$, иначе не активен.

6. Режим однократного импульса (One Pulse Mode)

6.1. Режим однократного импульса использует первые два канала таймера. Сигнал, поступающий на один канал, используется для запуска однократного импульса на другом канале после программной задержки.

6.2. Таймер в режиме *One Pulse* конфигурируется аналогично режиму ШИМ, однако счетчик запускается по внешнему импульсу и останавливается при первом возникновении события обновления.

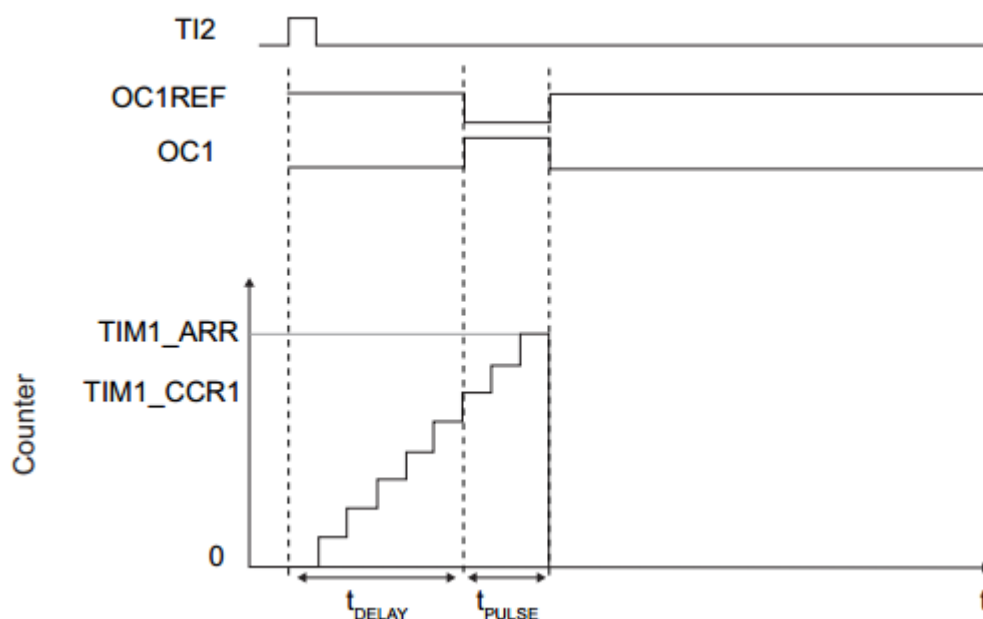


Рисунок 6 – Работа таймера в режиме One Pulse

6.3. Задержка и длительность выходного сигнала рассчитываются по следующим формулам:

$$\text{Задержка} = \frac{\text{Pulse}}{\left(\frac{\text{TIMx_CLK}}{\text{Prescaler} + 1} \right)}$$

$$\text{Длительность} = \frac{\text{Period} - \text{Pulse}}{\left(\frac{\text{TIMx_CLK}}{\text{Prescaler} + 1} \right)}$$

7. Режим энкодера

7.1. Когда таймер работает в режиме энкодера, значение регистра счетчика таймера TIMx_CNT увеличивается (уменьшается) по фронту сигналов на входных каналах.

7.2. Существуют два режима захвата. В режиме X2 счетчик таймера увеличивается/уменьшается на каждом фронте только одного канала (T1 или T2). В режиме X4 счетчик таймера обновляется на каждом фронте обоих каналов. Направление конфигурируется в регистре TIMx_DIR .

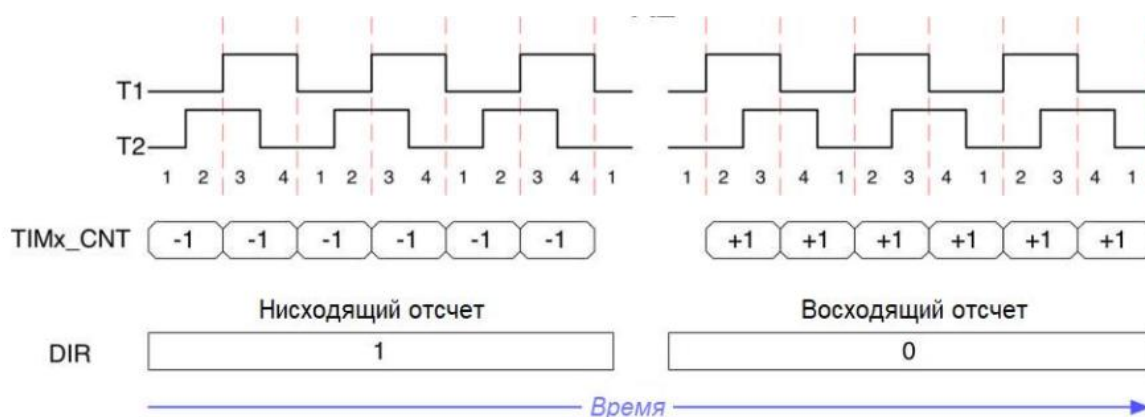


Рисунок 7 – Пример работы таймера в режиме энкодера X2

Часть 4. Продвинутое таймеры

1. Два таймера TIM1 и TIM8 являются 16-разрядными продвинутыми таймерами (Advanced Control Timers).

2. В отличие от таймеров общего назначения, продвинутые таймеры могут иметь до 6-ти каналов, а также дополнительные комплементарные выходы.

3. Продвинутый таймер имеет счетчик повторений, который хранится в регистре *TIMx_RCR*. Счетчик повторений позволяет сократить количество событий UE, как показано на рисунке 5.

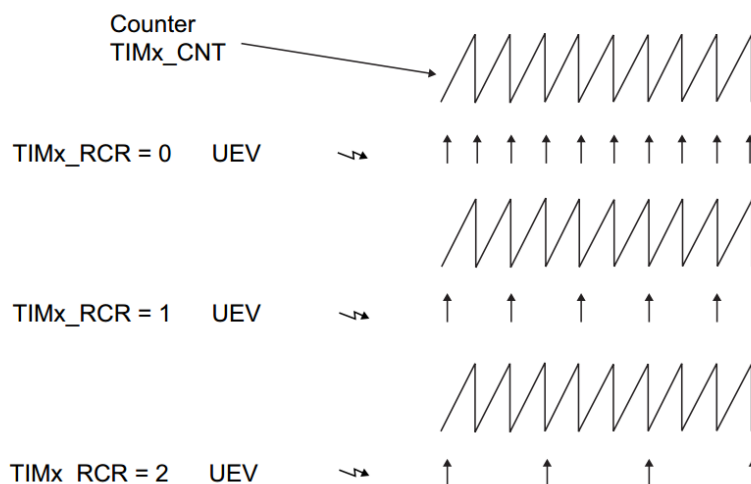


Рисунок 8 – Генерация события обновления при разных значениях счетчика повторений.

4. Продвинутые таймеры имеют два вывода сброса (*Break input*). Функция сброса таймера защищает силовые ключи, управляемые ШИМ-сигналами, генерируемыми таймерами расширенного управления. На выводы сброса таймера подается сигналы отказа силовых каскадов и 3-фазных инверторов. При активации цепь сброса таймера отключает выводы каналов таймера и устанавливает их в predetermined безопасное состояние.



Порядок выполнения работы

Часть 1. Создание проекта с использованием библиотеки HAL.

1. Создайте проект для отладочной платы ST NUCLEO STM32H745ZI-Q и подключите фреймворка HAL.



Создание проекта и конфигурация библиотеки HAL рассмотрено в ЛР6.

Способ 1: За основу проекта можно взять проект из ЛР6: сделайте копию и удалите исходные коды приложений в папках *src/cm4app* и *src/cm7app*.

Способ 2: Использовать шаблон проекта из папки *Resources*.

Часть 2. Генерация импульса с помощью таймера

1. Решим задачу создания одиночного импульса длиной 3 секунды на выводе МК с помощью аппаратного таймера на выводе PB0 (зеленый светодиод) и PC8. Оба вывода внутри МК подключены к каналу 3 таймера TIM3. Запуск импульса будем производить по нажатию кнопки USER.

2. Для решения будем воспользуемся режимом ШИМ таймера. В качестве периода ШИМ выберем требуемую длину импульса и запустим таймер с коэффициентом заполнения 100%. Обработаем прерывание о готовности импульса ШИМ и остановим таймер, чтобы завершить генерацию сигнала.

Какие еще режимы таймера можно использовать для решения задачи генерации импульса фиксированной длины?

3. Для начала создадим вспомогательный модуль для конфигурации контроллера EXTI для вызова по событию нажатия кнопки обработчика. Добавьте в папку `src/cm7app` следующий код и изучите его.

Листинг 1: `cm7app/key_button.h`

```
1  #pragma once
2
3  /* Обработка кнопки KeyButton по внешнему прерыванию
4   Для того чтобы использовать этот драйвер необходимо
5   1) Определить параметры кнопки с помощью макроопределений в этом файле
6   2) В обработчике соответствующей линии EXTI и вызвать
7   HAL_GPIO_EXTI_IRQHandler(KEY_BUTTON_PIN);
8   3) Переопределить weak-функцию void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) и вызвать в
9   ней обработчик нажатия на кнопку */
10 #define KEY_BUTTON_PIN GPIO_PIN_13
11 #define KEY_BUTTON_PORT GPIOC
12 #define KEY_BUTTON_CLK_ENABLE __HAL_RCC_GPIOC_CLK_ENABLE
13 #define KEY_BUTTON_IRQn EXTI15_10_IRQn
14
15 void Key_Button_EXTI_Init();
```

Листинг 2: `cm7app/key_button.c`

```
1  #include "key_button.h"
2  #include <stm32h7xx_hal.h>
3
4  void Key_Button_Init() {
5      KEY_BUTTON_CLK_ENABLE();
6      GPIO_InitTypeDef GPIO_InitStruct = {0};
7      GPIO_InitStruct.Pin = KEY_BUTTON_PIN;
8      GPIO_InitStruct.Pull = GPIO_PULLDOWN;
9      GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
10     HAL_GPIO_Init(KEY_BUTTON_PORT, &GPIO_InitStruct);
11     HAL_NVIC_SetPriority(KEY_BUTTON_IRQn, 0, 0);
12     HAL_NVIC_EnableIRQ(KEY_BUTTON_IRQn);
13 }
```

Вы можете заметить, что этот модуль похож на примитивную библиотеку, которая конфигурируется статически (во время компиляции) и поддерживает всего одну сущность – сигнал одной кнопки. Это отличает её от драйверов библиотеки HAL, которые поддерживают произвольное количество сущностей и конфигурируются во время выполнения.

4. Далее добавьте еще один вспомогательный модуль с функцией `assert_hal_status` для проверки статуса вызова функций библиотеки HAL. Изучите представленный код.

Листинг 3: `cm7app/error_handler.h`

```
1  #include <stm32h7xx_hal.h>
2
3  /** Функция аварийного завершения - остановка программы в цикле мигания красным
4   светодиодом */
5  void error_freeze() __attribute__((__noreturn__));
6
7  /** Проверка статуса HAL, с аварийным завершением в случае ошибки */
8  void assert_hal_status(HAL_StatusTypeDef status);
```

Листинг 4: `cm7app/error_handler.c`

```
1  #include "error_handler.h"
2  #include "hal_helpers.h"
3  #include "stm32h7xx_ll_utils.h"
4  #include <led.h>
5  #include <stdio.h>
6  #include <vterm.h>
7
8  void error_freeze() {
```

```

9   if (CoreDebug->DHCSR & 1) __BKPT(); // Если выполняется отладка, то прерываемся здесь
10  led_enable(led_red);
11  while (1) {
12      led_toggle(led_red);
13      LL_mDelay(250);
14  }
15  }
16
17  void assert_hal_status(HAL_StatusTypeDef status) {
18      if (status != HAL_OK) {
19          printf("\n\rERROR: %s", hal_status_to_string(status));
20          error_freeze();
21      }
22  }
23
24  /** Функция для обработки assert() из стандартной библиотеки */
25  void __assert_func(const char *file, int line, const char *func, const char *failedexpr) {
26      printf("\r\nAssertion \"%s\" failed in %s at %s:%d", failedexpr, func, file, line);
27      error_freeze();
28  }

```

5. Добавьте файлы с функциями для инициализации, запуска, остановки и деинициализации таймера, предназначенного для генерации сигнала.

5.1. Добавьте в проект следующие файлы.

Листинг 5: cm7app/tim_pulse.h



```

1  #pragma once
2  #include <stm32h7xx_hal.h>
3
4  #define TIMp          TIM3
5  #define TIMp_CHANNEL  TIM_CHANNEL_3
6  #define TIMp_PRESCALER (64000-1) /* 1 КГц */
7  #define TIMp_PERIOD    (3000 -1) /* UE каждые 3 с */
8  #define TIMp_CLK_ENABLE __HAL_RCC_TIM3_CLK_ENABLE
9  #define TIMp_IRQn      TIM3_IRQn
10 #define TIMp_IRQHandler TIM3_IRQHandler
11
12 // Вывод PB0 - зеленый светодиод
13 #define TIMp_CH_PORT    GPIOB
14 #define TIMp_CH_PIN     GPIO_PIN_0
15 #define TIMp_CH_CLK_ENABLE __HAL_RCC_GPIOB_CLK_ENABLE
16 #define TIMp_CH_GPIO_AF GPIO_AF2_TIM3
17
18 // Дополнительный Вывод PC8
19 #define TIMp_ECH_PORT    GPIOC
20 #define TIMp_ECH_PIN     GPIO_PIN_8
21 #define TIMp_ECH_CLK_ENABLE __HAL_RCC_GPIOC_CLK_ENABLE
22 #define TIMp_ECH_GPIO_AF GPIO_AF2_TIM3
23
24 extern TIM_HandleTypeDef hTimPulse;
25
26 /** Инициализация таймера для ШИМ генерации в режиме IT*/
27 void Tim_Pulse_Init();
28
29 /** Запуск таймера */
30 void Tim_Pulse_Start();
31
32 /** Остановка таймера */
33 void Tim_Pulse_Stop();
34
35 /** Деинициализация таймера */
36 void Tim_Pulse_Disable();

```

Листинг 6: cm7app/tim_pulse.c



```

1  #include "tim_pulse.h"

```

```

2  #include "error_handler.h"
3
4  TIM_HandleTypeDef hTimPulse;
5
6  void Tim_Pulse_Init() {
7      // Инициализация таймера в режим ШИМ
8      hTimPulse.Instance = TIMp;
9      hTimPulse.Init.Period = TIMp_PERIOD;
10     hTimPulse.Init.Prescaler = TIMp_PRESCALER;
11     hTimPulse.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
12     hTimPulse.Init.CounterMode = TIM_COUNTERMODE_UP;
13     hTimPulse.Init.RepetitionCounter = 0;
14     HAL_StatusTypeDef status = HAL_TIM_PWM_Init(&hTimPulse);
15     assert_hal_status(status);
16
17     // Инициализация выхода таймера
18     TIM_OC_InitTypeDef sOCConfig = {0};
19     sOCConfig.OCMode = TIM_OCMode_PWM2;
20     sOCConfig.OCpolarity = TIM_OCPolarity_HIGH;
21     sOCConfig.Pulse = 0; // коэффициент заполнения шим D = 100%
22     sOCConfig.OCIdleState = TIM_OCIDLESTATE_RESET;
23     sOCConfig.OCNPolarity = TIM_OCNPolarity_LOW;
24     sOCConfig.OCNIdleState = TIM_OCNIDLESTATE_RESET;
25     status = HAL_TIM_PWM_ConfigChannel(&hTimPulse, &sOCConfig, TIMp_CHANNEL);
26     assert_hal_status(status);
27 }
28
29 void Tim_Pulse_Start() {
30     if (HAL_TIM_GetChannelState(&hTimPulse, TIMp_CHANNEL) == HAL_TIM_CHANNEL_STATE_READY) {
31         HAL_StatusTypeDef status = HAL_TIM_PWM_Start_IT(&hTimPulse, TIMp_CHANNEL);
32         assert_hal_status(status);
33     }
34 }
35
36 void Tim_Pulse_Stop() {
37     HAL_StatusTypeDef status = HAL_TIM_PWM_Stop_IT(&hTimPulse, TIMp_CHANNEL);
38     assert_hal_status(status);
39 }
40
41 void Tim_Pulse_Disable() {
42     HAL_TIM_PWM_DeInit(&hTimPulse);
43 }
44
45 /**** Функции обратного вызова библиотеки HAL *****/
46 /** Переопределенные ниже функции обрабатывают любые PWM таймеры и обычно помещаются в
47     отдельный *msp.c
48     * файл В нашей программе только один PWM таймер, поэтому мы поместили эти функции здесь.
49     */
50
51 void HAL_TIM_PWM_MspInit(TIM_HandleTypeDef* htim) {
52     if (htim == &hTimPulse) {
53         // Включаем тактирование
54         TIMp_CLK_ENABLE();
55         TIMp_CH_CLK_ENABLE();
56         TIMp_ECH_CLK_ENABLE();
57
58         // Вывод на светодиод
59         GPIO_InitTypeDef GPIO_InitStruct;
60         GPIO_InitStruct.Pin = TIMp_CH_PIN;
61         GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
62         GPIO_InitStruct.Pull = GPIO_PULLDOWN;
63         GPIO_InitStruct.Speed = GPIO_SPEED_LOW;
64         GPIO_InitStruct.Alternate = TIMp_CH_GPIO_AF;
65         HAL_GPIO_Init(TIMp_CH_PORT, &GPIO_InitStruct);
66
67         // Вывод PC8
68         GPIO_InitStruct.Pin = TIMp_ECH_PIN;
69         GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
70         GPIO_InitStruct.Pull = GPIO_PULLDOWN;
71         GPIO_InitStruct.Speed = GPIO_SPEED_LOW;

```

```

72     GPIO_InitStruct.Alternate = TIMp_ECH_GPIO_AF;
73     HAL_GPIO_Init(TIMp_ECH_PORT, &GPIO_InitStruct);
74
75     // Включаем прерывания
76     HAL_NVIC_SetPriority(TIMp_IRQn, 2, 0);
77     HAL_NVIC_EnableIRQ(TIMp_IRQn);
78 }
79 }
80
81 void HAL_TIM_PWM_MspDeInit(TIM_HandleTypeDef* htim) {
82     if (htim == &hTimPulse) {
83         HAL_GPIO_DeInit(TIMp_CH_PORT, TIMp_CH_PIN);
84         HAL_GPIO_DeInit(TIMp_ECH_PORT, TIMp_ECH_PIN);
85     }
86 }

```

5.2. Инициализация таймера состоит из двух этапов. Вначале вызывается функция HAL_TIM_PWM_Init(), которая, в свою очередь, вызывает функцию HAL_TIM_PWM_MspInit(). В последней функции конфигурируются выводы GPIO в режим альтернативной функции для подключения к таймеру, демаскируется прерывание таймера и задается его приоритет.

5.3. Обратите внимание, что возможность подключения того или иного вывода (пина) к определённому каналу таймера задокументирована в [6].

5.4. На втором этапе конфигурируется работа канала таймера и параметры ШИМ с помощью HAL_TIM_PWM_ConfigChannel().

5.5. Таймер запускается в режиме прерываний с помощью функции HAL_TIM_PWM_Start_IT(). Для остановки таймера используется соответствующая функция HAL_TIM_PWM_Stop_IT().

5.6. Обратите внимание на макроопределения в файлах key_button.h и tim_pulse.h содержат «адреса» всех ресурсов, необходимых для реализации функций. Такой подход распространён при написании программ для микроконтроллеров и позволяет быстро адаптировать код программы при изменении ресурса. Например, можно быстро заменить таймер TIM3 на TIM4.

6. Добавьте в проект остальные файлы, реализующие основной функционал.

Листинг 7: cm7app/main.h



```

1  #pragma once
2  #include <stm32h7xx_hal.h>
3  #include <led.h>
4  #include <vterm.h>
5  #include "error_handler.h"
6  #include "key_button.h"
7  #include "tim_measure.h"
8  #include "tim_pulse.h"
9
10 #define VTERM_SPEED 115200

```

Листинг 8: cm7app/main.c



```

1  #include "main.h"
2  #include <stdio.h>
3
4  int main() {
5      printf("\r\nStart...\r\n");
6      HAL_Init();
7      Key_Button_EXTI_Init();
8      Tim_Pulse_Init();
9      Tim_Measure_Init();
10     Tim_Measure_Start();
11     printf("\r\nPress B1 to generate pulse on PC8 and green led\r\n");
12     __NOP();
13     while (1);

```

```

14     return 0;
15 }
16
17 /**** Функции обратного вызова библиотеки HAL *****/
18
19 void HAL_MspInit() {
20     __enable_irq();
21     HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_0);
22     led_enable(led_all);
23     led_off(led_all);
24     vterm_init(VTERM_SPEED);
25 }
26
27 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
28     if (GPIO_Pin == KEY_BUTTON_PIN) {
29         Tim_Pulse_Start();
30     }
31 }
32
33 void HAL_TIM_PWM_PulseFinishedCallback(TIM_HandleTypeDef *htim) {
34     if (htim == &hTimPulse) {
35         Tim_Pulse_Stop(hTimPulse);
36     }
37 }
38
39 void HAL_TIM_ErrorCallback(TIM_HandleTypeDef *htim) { (void) htim; error_freeze(); }

```

Листинг 9: cm7app/it.c



```

1  #include "main.h"
2
3  void SysTick_Handler() { HAL_IncTick(); }
4
5  void EXTI15_10_IRQHandler(void) {
6      HAL_GPIO_EXTI_IRQHandler(KEY_BUTTON_PIN);
7  }
8
9  void TIMP_IRQHandler() {
10     HAL_TIM_IRQHandler(&hTimPulse);
11 }

```

6.1. Файл `main.h` содержит общие константы, макроопределения и подключение заголовочных файлов. Далее этот файл подключается в файлах `main.c` и `it.c` с основной логикой программы. В фазах `error_handler.h/c` помещены все функции для обработки ошибок, а в файле `it.c` – обработчики задействованных прерываний.

В файле `main.c` находится функция `HAL_MspInit()`, в которой выполняется инициализация общих ресурсов – светодиодных индикаторов и вывода в терминал. В настоящем проекте Msp-функции распределены по отдельным файлам (единый Msp-файл отсутствует).

В функции `main()` вызываются функции инициализации: библиотеки HAL, обработчика кнопки и таймера.

Дальнейшая логика работы реализована в переопределённых функциях обратного вызова:

- `HAL_GPIO_EXTI_Callback()` – вызывается по прерыванию EXTI; в нашем случае - по нажатию кнопки USER. В обработчике запускается таймер;
- `HAL_TIM_PWM_PulseFinishedCallback()` – вызывается по завершению пульса (периода ШИМ сигнала). Обработчик останавливает работу таймера;
- `HAL_TIM_ErrorCallback()` – вызывается при появления флагов ошибок таймера.

7. Запустите программу и убедитесь, что по нажатию кнопки загорается и гаснет зеленый светодиод.

Часть 3. Измерение импульса с помощью таймера.

1. Доработаем программу так, чтобы измерить сформированный импульс с помощью другого таймера. Для этого соединим вывод PC8 (выходной сигнал) с выводом PE5, который может быть подключен к первому каналу таймера TIM15.

2. Отключите отладочную плату от компьютера и установите перемычку согласно рисунку 9.

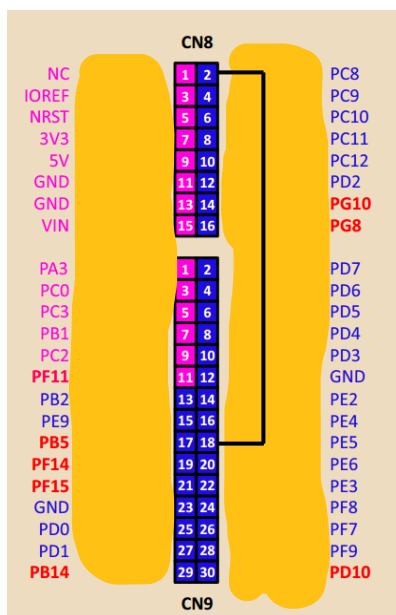


Рисунок 9 – Соединение вывода PC8 (канал TIM3:3) с выводом PE5 (канал TIM15:1) в отладочной плате ST Nucleo-H745ZI-Q

3. Добавьте в папку `src/cm7app` файлы `tim_measure.h` и `tim_measure.c`, в которых реализовано измерение импульса с помощью таймера и контроллера DMA.

Листинг 10: `cm7app/tim_measure.h`

```
1  #pragma once
2  #include "stm32h7xx_hal.h"
3
4  // Конфигурация таймера
5  #define TIMm          TIM15
6  #define TIMm_CHANNEL  TIM_CHANNEL_1
7  #define TIMm_PRESCALER (64000-1) // 1 КГц
8  #define TIMm_PERIOD    (65535)
9  #define TIMm_CLK_ENABLE __HAL_RCC_TIM15_CLK_ENABLE
10 #define TIMm_IRQn      TIM15_IRQn
11 #define TIMm_IRQHandler TIM15_IRQHandler
12
13 // Конфигурация DMA
14 #define TIMm_DMAx      DMA2
15 #define TIMm_DMA_STREAM DMA2_Stream1
16 #define TIMm_DMA_CLK_ENABLE __HAL_RCC_DMA2_CLK_ENABLE
17 #define TIMm_DMA_IRQn  DMA2_Stream1_IRQn
18 #define TIMm_DMA_IRQHandler DMA2_Stream1_IRQHandler
19 #define TIMm_DMA_ID     TIM_DMA_ID_CC1
20 #define TIMm_DMA_REQUEST DMA_REQUEST_TIM15_CH1
21 #define TIMm_DMA_ACTIVE_CHANNEL HAL_TIM_ACTIVE_CHANNEL_1
22
23 // Входной пин PA0
24 #define TIMm_CH_PORT    GPIOE
```

```

25 #define TIMm_CH_PIN          GPIO_PIN_5
26 #define TIMm_CH_CLK_ENABLE  __HAL_RCC_GPIOE_CLK_ENABLE
27 #define TIMm_CH_GPIO_AF      GPIO_AF4_TIM15
28
29 extern TIM_HandleTypeDef hTimMeasure;
30
31 /** Инициализация таймера в режим Input Capture Compare */
32 void Tim_Measure_Init();
33
34 /** Запуск таймера в режиме DMA */
35 void Tim_Measure_Start();
36
37 /* Остановка таймера*/
38 void Tim_Measure_Stop();
39
40 /* Функция обработки, которую следует поместить в HAL_TIM_IC_CaptureCallback*/
41 void Tim_Measure_IC_Callback(TIM_HandleTypeDef* htim);
42
43 /** Возвращает измеренное время в миллисекундах */
44 uint16_t Tim_Measure_GetDiff();
45
46 /* Ожидание завершения измерения импульса. Возвращает 1, если импульс был измерен
47    Повторный вызов, после успешного измерения будет возвращать 0 */
48 int Tim_Measure_Wait_Once(uint32_t timeout);
49
50 /** Деинициализация таймера */
51 void Tim_Measure_Deinit();

```

Листинг 11: app/tim_measure.c



```

1  #include "tim_measure.h"
2  #include "error_handler.h"
3
4  TIM_HandleTypeDef hTimMeasure;
5  static volatile int capture_done = 0;
6
7  static uint16_t captures[4]
8      __attribute__((aligned(16))); // __attribute__((section(".dma_buffer")));
9
10 void Tim_Measure_Init() {
11     /** Конфигурация в режим Input Compare */
12     HAL_StatusTypeDef status = HAL_OK;
13     hTimMeasure.Instance = TIMm;
14     hTimMeasure.Init.Period = TIMm_PERIOD;
15     hTimMeasure.Init.Prescaler = TIMm_PRESCALER;
16     hTimMeasure.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
17     hTimMeasure.Init.CounterMode = TIM_COUNTERMODE_UP;
18     hTimMeasure.Init.RepetitionCounter = 0;
19     status = HAL_TIM_IC_Init(&hTimMeasure);
20     assert_hal_status(status);
21
22     /** Конфигурация входного канала */
23     TIM_IC_InitTypeDef sICConfig = {0};
24     sICConfig.ICPolarity = TIM_INPUTCHANNELPOLARITY_BOTHEDGE;
25     sICConfig.ICSelection = TIM_ICSELECTION_DIRECTTI;
26     sICConfig.ICPrescaler = TIM_ICPSC_DIV1;
27     sICConfig.ICFilter = 0;
28     status = HAL_TIM_IC_ConfigChannel(&hTimMeasure, &sICConfig, TIMm_CHANNEL);
29     assert_hal_status(status);
30 }
31
32 void Tim_Measure_Deinit() {
33     HAL_StatusTypeDef status = HAL_TIM_IC_DeInit(&hTimMeasure);
34     assert_hal_status(status);
35 }
36
37 void Tim_Measure_Start() {
38     capture_done = 0;

```

```

39     HAL_StatusTypeDef status =
40         HAL_TIM_IC_Start_DMA(&hTimMeasure, TIMm_CHANNEL, (uint32_t *)captures, 2);
41     assert_hal_status(status);
42 }
43
44 void Tim_Measure_Stop() {
45     HAL_StatusTypeDef status = HAL_TIM_IC_Stop_DMA(&hTimMeasure, TIMm_CHANNEL);
46     assert_hal_status(status);
47 }
48
49 uint16_t Tim_Measure_GetDiff() {
50     if (captures[1] >= captures[0]) {
51         return captures[1] - captures[0];
52     } else {
53         return TIMm_PERIOD - captures[0] + captures[1];
54     }
55 }
56
57 void Tim_Measure_IC_Callback(TIM_HandleTypeDef *htim) {
58     if (htim == &hTimMeasure) {
59         if (htim->Channel == TIMm_DMA_ACTIVE_CHANNEL)
60             capture_done = 1;
61     }
62 }
63
64 int Tim_Measure_Wait_Once(uint32_t timeout) {
65     uint32_t time_limit = HAL_GetTick() + timeout;
66     while (capture_done == 0 && HAL_GetTick() < time_limit);
67     if (capture_done) {
68         capture_done = 0;
69         return 1;
70     };
71     return 0;
72 }
73
74 /**** Функции обратного вызова библиотеки HAL *****/
75 /** Переопределенные ниже функции обрабатывают любые IC-таймеры и обычно помещаются в
76     отдельный *msp.c
77     * В нашей программе только один PWM таймер, поэтому мы поместили эти функции здесь.
78     */
79
80 void HAL_TIM_IC_MspInit(TIM_HandleTypeDef *htim) {
81     if (htim == &hTimMeasure) {
82         HAL_StatusTypeDef status = HAL_OK;
83
84         TIMm_CLK_ENABLE();
85         TIMm_CH_CLK_ENABLE();
86         TIMm_DMA_CLK_ENABLE();
87
88         /* Конфигурация вывода на работу с таймером */
89         GPIO_InitTypeDef GPIO_InitStruct;
90         GPIO_InitStruct.Pin = TIMm_CH_PIN;
91         GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
92         GPIO_InitStruct.Pull = GPIO_PULLDOWN;
93         GPIO_InitStruct.Speed = GPIO_SPEED_LOW;
94         GPIO_InitStruct.Alternate = TIMm_CH_GPIO_AF;
95         HAL_GPIO_Init(TIMm_CH_PORT, &GPIO_InitStruct);
96
97         /* Инициализация DMA */
98         static DMA_HandleTypeDef hdma_tim;
99         hdma_tim.Instance = TIMm_DMA_STREAM;
100        hdma_tim.Init.Direction = DMA_PERIPH_TO_MEMORY;
101        hdma_tim.Init.PeriphInc = DMA_PINC_DISABLE;
102        hdma_tim.Init.MemInc = DMA_MINC_ENABLE;
103        hdma_tim.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD;
104        hdma_tim.Init.MemDataAlignment = DMA_MDATAALIGN_HALFWORD;
105        hdma_tim.Init.Mode = DMA_CIRCULAR;
106        hdma_tim.Init.Priority = DMA_PRIORITY_LOW;
107        hdma_tim.Init.Request = TIMm_DMA_REQUEST;
108        hdma_tim.Init.FIFOMode = DMA_FIFOMODE_DISABLE;

```

```

109     hdma_tim.Init.FIFOThreshold = DMA_FIFO_THRESHOLD_1QUARTERFULL;
110     hdma_tim.Init.MemBurst = DMA_MBURST_SINGLE;
111     hdma_tim.Init.PeriphBurst = DMA_PBURST_SINGLE;
112
113     /* Несколько указателей на DMA-дескрипторы в периферии должны указывать на один
114        и тот же дескриптор DMA. Существует только один канал для выполнения всех запросов
115        к DMA. */
116     __HAL_LINKDMA(&hTimMeasure, hdma[TIMm_DMA_ID], hdma_tim);
117
118     status = HAL_DMA_Init(hTimMeasure.hdma[TIMm_DMA_ID]);
119     assert_hal_status(status);
120
121     HAL_NVIC_SetPriority(TIMm_IRQn, 2, 0);
122     HAL_NVIC_SetPriority(TIMm_DMA_IRQn, 3, 0);
123     HAL_NVIC_EnableIRQ(TIMm_IRQn);
124     HAL_NVIC_EnableIRQ(TIMm_DMA_IRQn);
125 }
126 }
127
128 void HAL_TIM_IC_MspDeInit(TIM_HandleTypeDef *htim) {
129     if (htim == &hTimMeasure) {
130         HAL_GPIO_DeInit(TIMm_CH_PORT, TIMm_CH_PIN);
131     }
132 };

```

4. В файл `main.h` добавьте подключение:

```
#include "tim_measure.h"
```

5. Добавьте функции обработчики прерывания для нового таймера:

```

void TIMm_IRQHandler() { HAL_TIM_IRQHandler(&hTimMeasure); }
void TIMm_DMA_IRQHandler(void) { HAL_DMA_IRQHandler(hTimMeasure.hdma[TIMm_DMA_ID]); }

```

6. Изучите код:

6.1. Измерение длительности сигнала выполняется в режиме *Input Compare*, поэтому для конфигурации и запуска таймера используются функции `HAL_TIM_IC_Init()`, `HAL_TIM_IC_ConfigChannel()`, `HAL_TIM_IC_MspInit()`. Работа с таймером выполняется через контроллер DMA, поэтому для запуска и остановки измерений вызываются функции `HAL_TIM_IC_Start_DMA()` и `HAL_TIM_IC_Stop_DMA()`.

6.2. В функции `HAL_TIM_IC_MspInit()` выполняется конфигурация потока контроллера DMA. При каждом изменении фронта сигнала будет передаваться запрос контроллеру DMA. Получив этот запрос контроллер DMA скопирует значение из регистра захвата в указанный буфер. Подключении потока DMA к таймеру выполняется с помощью специального макроса `__HAL_LINKDMA()`. Также демаскируется прерывание для потока DMA, чтобы обработать данные после того как будут получено два значения.

6.3. В обработчике готовности приёма данных вычисляется продолжительность сигнала и устанавливается флаг готовности `capture_done`. Этот флаг используется в основном цикле программы контроля готовности данных для однократного вывода вновь измеренного расстояния.

7. В файле `main.c` определите функцию обратного вызова для события захвата таймера:

```
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef* htim) { Tim_Measure_IC_Callback(htim); }
```

8. Измените функцию `main()` так, чтобы в терминал выводилось сообщение об измеренном импульсе:

```

int main() {
    printf("\r\nStart...\r\n");
    HAL_Init();
    Key_Button_EXTI_Init();
    Tim_Pulse_Init();
}

```

```

Tim_Mesure_Init();
Tim_Mesure_Start();
printf("\r\nPress B1 to generate pulse on PC8 and mesure it on PE5\r\n");
__NOP();
while (1) {
    if (Tim_Mesure_Wait_Once(1000)) {
        int time_ms = Tim_Mesure_GetDiff();
        printf("\r\nMesured Pulse Time = %d\r\n", time_ms);
    } else {
        printf("\r\nwaiting...");
    }
}
return 0;
}

```

9. Выполните сборку программу и запустите её на микроконтроллере.

Откройте терминал *SerialMonitor*. Нажмите на кнопку USER (B1) и наблюдайте вывод сообщения об измеренном времени импульса после того, как зеленый светодиод потухнет.

Часть 4. Ультразвуковой датчик расстояния HC-SR04

1. Ознакомьтесь с документацией на ультразвуковой датчик расстояния HC-SR04 [4].

2. Схема работы датчика показана рисунке 10. Для измерения расстояния до объекта необходимо сформировать импульс продолжительностью не менее 10 мкс на входе Input. В ответ на выводе Echo будет сформирован сигнал продолжительностью от 100 мкс до 18 мс. В случае отсутствия какого-либо препятствия выходной импульс составляет приблизительно 36 мс.

Расстояние до объекта пропорционально длине ответного импульса и рассчитывается по формуле:

$$\text{Расстояние (см)} = (\text{Длина ответного импульса}) / 58.$$

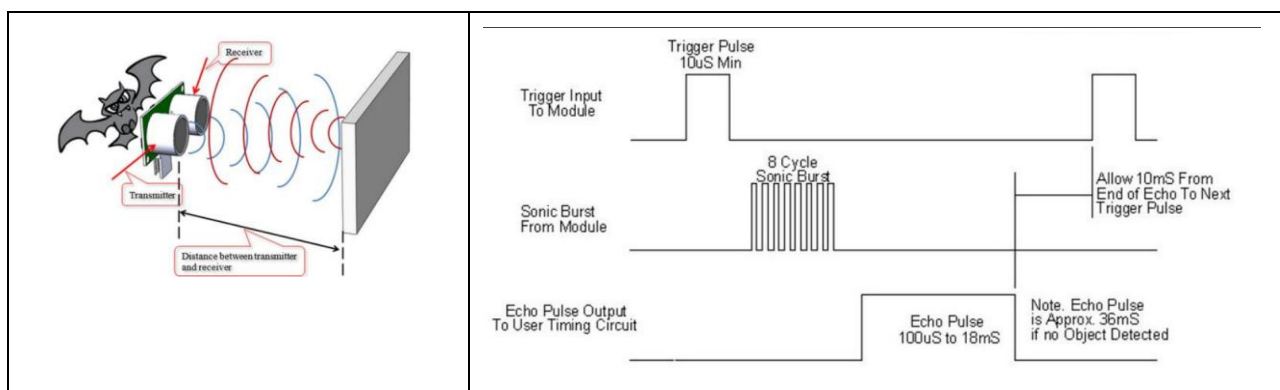
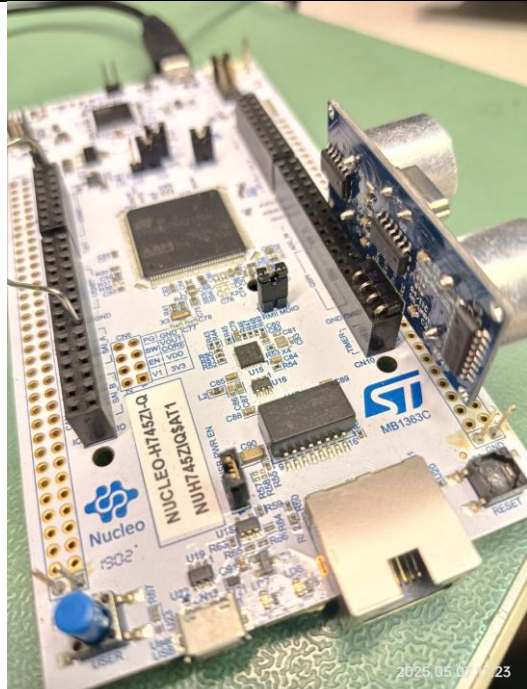
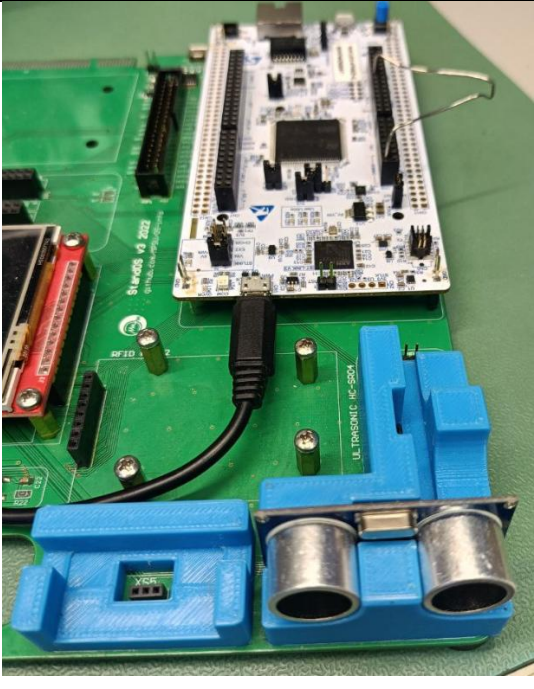


Рисунок 10 – Схема работы датчика HC-SR04

Практическое задание. Измерение расстояния с помощью датчика HC-SR04.

1. Выполните подключение датчика HC-SR04 согласно варианту.

Вариант 1	Вариант 2																																																																																																						
Подключение к CN10	Подключение к разъему H0																																																																																																						
<table><tr><td>VDDA</td><td>AVDD</td><td>1</td><td>2</td><td>D7</td><td>PG12</td></tr><tr><td>AGND</td><td>AGND</td><td>3</td><td>4</td><td>D6</td><td>PA8</td></tr><tr><td>GND</td><td>GND</td><td>5</td><td>6</td><td>D5</td><td>PE11</td></tr><tr><td>PC1</td><td>A6</td><td>7</td><td>8</td><td>D4</td><td>PE14</td></tr><tr><td>PC5</td><td>A7</td><td>9</td><td>10</td><td>D3</td><td>PE13</td></tr><tr><td>PA2</td><td>A8</td><td>11</td><td>12</td><td>D2</td><td>PG14</td></tr><tr><td>PG6</td><td>D26</td><td>13</td><td>14</td><td>D1</td><td>PB6</td></tr><tr><td>PB2</td><td>D27</td><td>15</td><td>16</td><td>D0</td><td>PB7</td></tr><tr><td>GND</td><td>GND</td><td>17</td><td>18</td><td>D42</td><td>PE8</td></tr><tr><td>PD13</td><td>D28</td><td>19</td><td>20</td><td>D41</td><td>PE7</td></tr><tr><td>PD12</td><td>D29</td><td>21</td><td>22</td><td>GND</td><td>GND</td></tr><tr><td>PD11</td><td>D30</td><td>23</td><td>24</td><td>D40</td><td>PE10</td></tr><tr><td>PE2</td><td>D31</td><td>25</td><td>26</td><td>D39</td><td>PE12</td></tr><tr><td>GND</td><td>GND</td><td>27</td><td>28</td><td>D38</td><td>PE6</td></tr><tr><td>PA0</td><td>Echo</td><td>29</td><td>30</td><td>D37</td><td>PE15</td></tr><tr><td>PB0</td><td>Trig</td><td>31</td><td>32</td><td>D36</td><td>PB10</td></tr><tr><td>PE0</td><td>VCC</td><td>33</td><td>34</td><td>D35</td><td>PB11</td></tr></table> CN10	VDDA	AVDD	1	2	D7	PG12	AGND	AGND	3	4	D6	PA8	GND	GND	5	6	D5	PE11	PC1	A6	7	8	D4	PE14	PC5	A7	9	10	D3	PE13	PA2	A8	11	12	D2	PG14	PG6	D26	13	14	D1	PB6	PB2	D27	15	16	D0	PB7	GND	GND	17	18	D42	PE8	PD13	D28	19	20	D41	PE7	PD12	D29	21	22	GND	GND	PD11	D30	23	24	D40	PE10	PE2	D31	25	26	D39	PE12	GND	GND	27	28	D38	PE6	PA0	Echo	29	30	D37	PE15	PB0	Trig	31	32	D36	PB10	PE0	VCC	33	34	D35	PB11	Trig - PD12 Echo – PD14
VDDA	AVDD	1	2	D7	PG12																																																																																																		
AGND	AGND	3	4	D6	PA8																																																																																																		
GND	GND	5	6	D5	PE11																																																																																																		
PC1	A6	7	8	D4	PE14																																																																																																		
PC5	A7	9	10	D3	PE13																																																																																																		
PA2	A8	11	12	D2	PG14																																																																																																		
PG6	D26	13	14	D1	PB6																																																																																																		
PB2	D27	15	16	D0	PB7																																																																																																		
GND	GND	17	18	D42	PE8																																																																																																		
PD13	D28	19	20	D41	PE7																																																																																																		
PD12	D29	21	22	GND	GND																																																																																																		
PD11	D30	23	24	D40	PE10																																																																																																		
PE2	D31	25	26	D39	PE12																																																																																																		
GND	GND	27	28	D38	PE6																																																																																																		
PA0	Echo	29	30	D37	PE15																																																																																																		
PB0	Trig	31	32	D36	PB10																																																																																																		
PE0	VCC	33	34	D35	PB11																																																																																																		
																																																																																																							

2. Разработайте программу для ядра Cortex-M7 со следующим поведением:

- 2.1. После запуска программа непрерывно с частотой 2 Гц измеряется текущее расстояние с помощью модуля HC-SR04. Работа с модулем должна выполняться с помощью таймеров.
- 2.2. Выполняется индикация светодиодами:
 - менее 10 см – красный; от 10 см до 20 см – желтый; более 20 см – зеленый
- 2.3. По нажатию на кнопку в терминал выводится текущее расстояние (в сантиметрах).



Вопросы для самостоятельной проверки

- 1) Что такое сигнал ШИМ, параметры «длительность импульса» и «коэффициент заполнения»?
- 2) Чем отличаются базовые таймеры от таймеров общего назначения?
- 3) Чем отличаются продвинутые (Advanced Control) таймеры от таймеров общего назначения?
- 4) Какие режимы синхронизации таймеров существуют?
- 5) Назовите и объясните назначение основных регистров базового таймера?
- 6) Какие функциональные режимы работы поддерживают таймеры общего назначения?
- 7) Как работает таймер в режиме ШИМ?
- 8) Как работает таймер в режиме входного захвата?
- 9) Как работает таймер в режиме сравнения-вывода?
- 10) Как работает таймер в режиме One Pulse Mode?
- 11) Как работает таймер в режиме энкодера?
- 12) Как работают таймера в режимах сна и остановки основного процессора?
- 13) Как рассчитывается частота события обновления таймера?
- 14) Какие действия выполняются при наступлении события обновления в таймере?
- 15) Каким образом может быть сконфигурирован входной канал таймера?
- 16) Какие источники тактирования таймеров существуют?
- 17) Что такое контроллер DMA и какие режимы работы он имеет?
- 18) Сколько DMA контроллеров имеется МК STM32 H745ZI-Q?
- 19) Сколько потоков имеют контроллеры DMA в МК STM32 H745ZI-Q?
- 20) Как коммутируется линия запроса DMA с периферийным блоком в библиотеке HAL?
- 21) Как коммутируется линия запроса DMA с периферийным блоком в МК?
- 22) Какие режимы ШИМ имеются в таймерах общего назначения STM32?
- 23) *Зачем нужен буфер в контроллере DMA?